

به نام خدا

طراحی الگوریتم‌ها

آرش شفیعی



الگوریتم‌های تقسیم و حل

الگوریتم‌های تقسیم و حل

- برای حل یک مسئله به روش‌های متنوعی می‌توان الگوریتم طراحی کرد.
- الگوریتم مرتب‌سازی درجی یک الگوریتم ساده است که به روش افزایشی با مرتب‌سازی زیر آرایه‌های کوچک‌تر آرایه آغاز می‌شود و در نهایت کل آرایه را مرتب می‌کند. در واقع به ازای هر عنصر $A[i]$ ، این عنصر در مکان مناسب خود در زیر آرایه مرتب شده $A[1 : i-1]$ قرار می‌گیرد.

الگوریتم‌های تقسیم و حل

- در این قسمت با روشی دیگر برای حل مسئله‌های محاسباتی آشنا می‌شویم، که به آن روش تقسیم و حل¹ (تقسیم و غلبه) گفته می‌شود و الگوریتم‌هایی که از این روش استفاده می‌کنند، در دسته الگوریتم‌های تقسیم و حل قرار می‌گیرند.
- از روش تقسیم و حل برای حل مسئله مرتب‌سازی استفاده می‌کنیم و زمان اجرای آن را محاسبه می‌کنیم.
- خواهیم دید که با استفاده از این روش، مسئله مرتب‌سازی در زمان کمتری نسبت به الگوریتم مرتب‌سازی درجی حل می‌شود.

¹ divide and conquer method

الگوریتم‌های تقسیم و حل

- بسیاری از الگوریتم‌های کامپیوتری بازگشتی¹ هستند. در یک الگوریتم بازگشتی، برای حل یک مسئله با یک ورودی معین، خود الگوریتم با ورودی‌های کوچکتر فراخوانی می‌شود.
- برای مثال، برای به دست آوردن فاکتوریل عدد n کافی است فاکتوریل عدد $n-1$ را فراخوانی کنیم.
- به الگوریتم‌هایی که ورودی مسئله را تقسیم می‌کنند و به طور بازگشتی الگوریتم را برای قسمت‌های تقسیم شده فراخوانی می‌کنند، الگوریتم‌های تقسیم و حل گفته می‌شود.

¹ recursive

الگوریتم‌های تقسیم و حل

- به عبارت دیگر یک الگوریتم تقسیم و حل یک مسئله را به چند زیر مسئله تقسیم می‌کند که مشابه مسئله اصلی هستند و الگوریتم را برای زیر مسئله‌ها فراخوانی می‌کند و سپس نتایج به دست آمده از زیر مسئله‌ها را با هم ترکیب می‌کند تا نتیجه نهایی برای مسئله اصلی به دست آید.
- معمولاً پس از شکسته شدن یک مسئله به زیر مسئله‌ها، زیر مسئله‌هایی به دست می‌آیند که می‌توانند دوباره شکسته شوند و این روند تا جایی ادامه پیدا می‌کند که مسئله امکان شکسته شدن نداشته باشد. وقتی مسئله امکان شکسته شدن نداشته باشد، حالت پایه¹ به دست می‌آید که حل مسئله در حالت پایه به سادگی امکان پذیر است.

¹ base case

الگوریتم‌های تقسیم و حل

- یک الگوریتم تقسیم و حل از سه مرحله زیر تشکیل شده است.
۱. تقسیم¹ : مسئله به چند زیر مسئله که نمونه‌های کوچکتر مسئله اصلی هستند تقسیم می‌شود.
 ۲. حل یا غلبه² : زیر مسئله‌ها به صورت بازگشتی حل می‌شوند.
 ۳. ترکیب³ : زیر مسئله‌های حل شده با یکدیگر ترکیب می‌شوند تا جواب مسئله اصلی به دست بیاید.

¹ divide

² conquer

³ combine

- الگوریتم مرتب‌سازی ادغامی¹ در دسته الگوریتم‌های تقسیم و حل قرار می‌گیرد. با شروع از آرایه $A[1:n]$ ، در هر مرحله یکی از زیر آرایه‌های $A[p:r]$ مرتب می‌شود و سپس این زیر آرایه‌ها با یکدیگر ادغام می‌شوند تا آرایه اصلی مرتب شود. برای هر یک از زیر آرایه‌ها، الگوریتم مرتب‌سازی ادغامی فراخوانی می‌شود و به همین نحو، آن زیر آرایه‌ها تقسیم شده و به روش بازگشتی مرتب می‌شوند.

¹ merge sort

مرتب‌سازی ادغامی

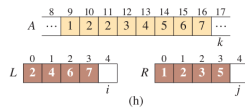
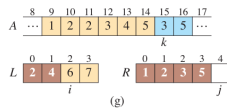
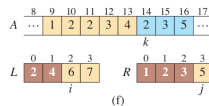
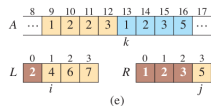
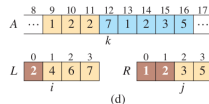
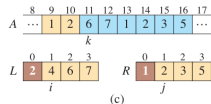
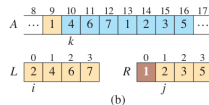
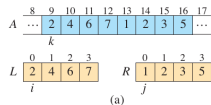
- مراحل انجام مرتب‌سازی ادغامی به صورت زیر است :

۱. تقسیم : آرایه $A[p:r]$ به دو زیرآرایه مساوی تقسیم می‌شود. اگر q وسط p و r باشد، آنگاه دو آرایه به دست آمده عبارتند از $A[p:q]$ و $A[q+1:r]$. در مرحله اول p برابر با 1 و r برابر است با n .
۲. حل : الگوریتم به صورت بازگشتی برای دو زیر آرایه $A[p:q]$ و $A[q+1:r]$ فراخوانی می‌شود.
۳. ترکیب : با ادغام دو آرایه $A[p:q]$ و $A[q+1:r]$ که هر دو مرتب شده هستند، آرایه مرتب شده $A[p:r]$ به دست می‌آید.

- این الگوریتم به طور بازگشتی فراخوانی می‌شود تا به حالت پایه برسیم. در حالت پایه، آرایه به دست آمده شامل تنها یک عنصر است که در این حالت آرایه نیاز به مرتب‌سازی ندارد. در واقع هنگامی به حالت پایه می‌رسیم که p برابر با r باشد.
- در مرحله ادغام، با فرض اینکه دو آرایه به دست آمده مرتب شده هستند، دو آرایه باید به نحوی با یکدیگر ترکیب شوند که آرایه به دست آمده مرتب شده باشد.

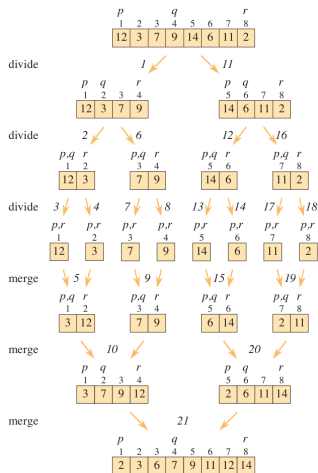
مرتب‌سازی ادغامی

- یک مثال از ادغام دو زیر آرایه در شکل زیر نشان داده شده است.



مرتب‌سازی ادغامی

- یک مثال مرتب‌سازی ادغامی در شکل زیر نشان داده شده‌است.



- الگوریتم مرتب‌سازی ادغامی به صورت زیر است.

Algorithm Merge Sort

```
function MERGE-SORT(A, p, r)
1: if p >= r then ▷ zero or one element?
2:   return
3: q = ⌊ (p+r)/2 ⌋ ▷ midpoint of A[p:r]
4: Merge-Sort (A, p, q) ▷ recursively sort A[p:q]
5: Merge-Sort (A, q+1, r) ▷ recursively sort A[q+1:r]
6: Merge (A, p, q, r) ▷ Merge A[p:q] and A[q+1:r] into A[p:r].
```

- برای ادغام دو زیرآرایه از الگوریتم زیر استفاده می‌کنیم.

Algorithm Merge Sort

```
function MERGE(A, p, q, r)
1: nl = q - p + 1 ▷ length of A[p:q]
2: nr = r - q ▷ length of A[q+1 : r]
3: let L[ 0 : nl - 1 ] and R[ 0 : nr - 1 ] be new arrays
4: for i = 0 to nl - 1 do ▷ copy A[p:q] into L[0:nl - 1]
5:   L[i] = A[p+i]
6: for j = 0 to nr - 1 do ▷ copy A[q+1:r] into L[0:nr - 1]
7:   R[j] = A[q + j + 1]
```

Algorithm Merge Sort

```

function MERGE(A, p, q, r)
8: i = 0  ▷ i indexes the smallest remaining element in L
9: j = 0  ▷ j indexes the smallest remaining element in R
10: k = p  ▷ k indexes the location in A to fill
    ▷ As long as each of the arrays L and R contains an unmerged element,
    copy the smallest unmerged element back into A[p : r].
11: while i < nl and j < nr do
12:   if L[i] <= R[j] then
13:     A[k] = L[i]
14:     i = i + 1
15:   else
16:     A[k] = R[j]
17:     j = j + 1
18:   k = k + 1

```

Algorithm Merge Sort

function MERGE(A, p, q, r)

▷ Having gone through one of L and R entirely, copy the remainder of the other to the end of A[p:r]

19: while i < nl do

20: A[k] = L[i]

21: i = i + 1

22: k = k + 1

23: while j < nr do

24: A[k] = R[j]

25: j = j + 1

26: k = k + 1

- در حلقهٔ تکرار الگوریتم ادغام، در هر تکرار یکی از عناصر در آرایهٔ A کپی می‌شوند و در کل تا پایان الگوریتم n عنصر در آرایه کپی می‌شوند، پس زمان اجرای این الگوریتم $\Theta(n)$ است.

- وقتی یک مسئله به صورت بازگشتی طراحی می‌شود، زمان اجرای آن را نیز معمولاً با استفاده از معادلات بازگشتی¹ به دست می‌آوریم. در این معادلات بازگشتی، زمان اجرای یک الگوریتم با ورودی اندازه n توسط زمان اجرای همان الگوریتم با ورودی‌هایی از اندازه‌های کوچک‌تر به دست می‌آید. روش‌های متعددی برای حل مسائل بازگشتی وجود دارند که می‌توان از آنها استفاده کرد.

¹ recurrence equation

- به طور کلی اگر فرض کنیم زمان اجرای یک الگوریتم برای ورودی با اندازه n برابر با $T(n)$ باشد و توسط روش تقسیم و حل مسئله مورد نظر به a زیر مسئله تقسیم شود که اندازه ورودی هر کدام n/b باشد، آنگاه به زمان $aT(n/b)$ برای حل مسئله نیاز داریم.
- همچنین اگر به زمان $D(n)$ برای تقسیم مسئله به زیر مسئله‌ها و به زمان $C(n)$ برای ادغام زیر مسئله‌ها نیاز داشته باشیم، آنگاه این زمان‌ها به زمان مورد نیاز برای حل مسئله افزوده می‌شوند.

- فرض کنید در حالت پایه، یعنی وقتی اندازه ورودی از یک مقدار معین کوچکتر است، اجرای برنامه در زمان ثابت انجام شود، یعنی زمان اجرای برنامه در حالت پایه به اندازه ورودی n بستگی نداشته باشد.
- در حالت کلی زمان اجرای یک الگوریتم تقسیم و حل را می‌توانیم با استفاده از رابطه بازگشتی زیر بنویسیم.

$$T(n) = \begin{cases} \Theta(1) & \text{اگر } n < n_0 \\ D(n) + aT(n/b) + C(n) & \text{در باقی حالات} \end{cases}$$

- حال زمان اجرای الگوریتم مرتب‌سازی ادغامی را در بدترین حالت به ازای یک آرایه با طول n تحلیل می‌کنیم.
۱. تقسیم : تقسیم کردن آرایه به دو قسمت در زمان ثابت انجام می‌شود، بنابراین داریم $D(n) = \Theta(1)$.
 ۲. حل : در مرحله حل از دو آرایه با اندازه $n/2$ به صورت بازگشتی استفاده می‌کنیم بنابراین زمان مورد نیاز در این مرحله برابر است با $2T(n/2)$. توجه کنید که ممکن است آرایه بر دو بخش پذیر نباشد، اما معمولاً در تحلیل الگوریتم از توابع کف و سقف صرف نظر می‌کنیم، چرا که تأثیری در تحلیل الگوریتم نمی‌گذارند.
 ۳. ترکیب : در این مرحله برای ادغام دو آرایه، جهت تولید یک آرایه با طول n به زمان $\Theta(n)$ نیاز داریم، بنابراین داریم $C(n) = \Theta(n)$.

- بنابراین در مجموع زمان اجرای الگوریتم مرتب‌سازی ادغامی به صورت زیر است :

$$T(n) = 2T(n/2) + \Theta(n)$$

- با حل این معادله بازگشتی می‌توان به دست آورد $T(n) = \Theta(n \lg n)$ ، بنابراین زمان مورد نیاز برای اجرای الگوریتم مرتب‌سازی ادغامی از مرتب‌سازی درجی بهتر است.

مرتب‌سازی ادغامی

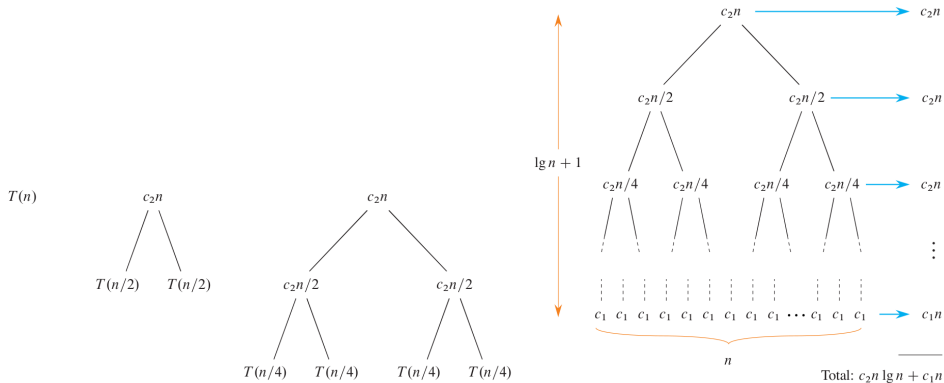
- حال برای اینکه بدون حل معادله بازگشتی، زمان اجرای به دست آمده را درک کنیم، می‌توانیم الگوریتم را به صورت زیر تحلیل کنیم.
- برای سادگی فرض می‌کنیم طول آرایه ورودی برابر با n بوده و n توانی از ۲ است. با این ساده‌سازی همیشه با تقسیم n بر ۲ یک عدد صحیح به دست می‌آید.
- زمان اجرای الگوریتم را به صورت زیر می‌نویسیم.

$$T(n) = \begin{cases} c_1 & \text{اگر } n = 1 \\ 2T(n/2) + c_2n & \text{اگر } n > 1 \end{cases}$$

- در اینجا c_1 زمان اجرای الگوریتم است هنگامی که طول ورودی ۱ باشد و c_2 مضرب ثابتی است که برای تقسیم و ادغام آرایه با طول n نیاز داریم.

مرتب سازی ادغامی

- شکل های زیر تقسیم این مسئله را به زیر مسئله ها و تحلیل زمان زیر مسئله ها را نشان می دهد.



- مقدار c_2n در ریشهٔ این درخت در واقع زمان مورد نیاز برای تقسیم و ادغام را نشان می‌دهد، هنگامی که اندازهٔ مسئله برابر است با n . دو زیر درخت در سطح ۱ این درخت زمان‌های مورد نیاز را وقتی اندازهٔ ورودی $n/2$ است نشان می‌دهند. هزینه مورد نیاز برای تقسیم و ادغام هر کدام از این زیر درخت‌ها برابر است با $c_2n/2$ و مجموعه این هزینه‌ها برای دو زیر درخت برابر است با c_2n .
- چنانچه این محاسبات را ادامه دهیم، به این نتیجه می‌رسیم که هزینه تقسیم و ادغام برای هر یک از سطوح درخت برابر است با c_2n .

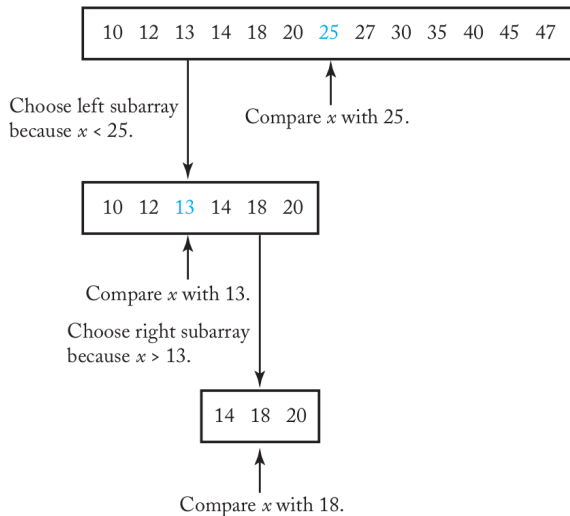
- سطح آخر، یعنی سطحی که برگ‌های درخت در آن قرار دارد، حالت پایه را نشان می‌دهد که در این حالت زمان اجرای هر یک از زیر آرایه‌ها برابر است با c_1 و چون تعداد n زیر آرایه با طول ۱ داریم، زمان اجرا برای کل زیر آرایه‌ها برابر است با $c_1 n$.
- از آنجایی که این درخت در هر مرحله به دو بخش تقسیم می‌شود، تعداد سطوح درخت برابر است با $\lg n + 1$.
- بنابراین زمان کل اجرای الگوریتم برابر است با $c_2 n \lg n + c_1 n$.
- می‌توانیم با استفاده از تحلیل مجانبی بنویسیم $T(n) = \Theta(n \lg n)$.

- برای جستجوی یک مقدار در یک آرایه باید همهٔ عناصر آرایه را یک به یک بررسی کنیم. این جستجو برای یک آرایه با n عنصر در زمان $O(n)$ انجام می‌شود.
- حال فرض می‌کنیم می‌خواهیم یک مقدار را در یک آرایه مرتب شده پیدا کنیم.
- برای این کار می‌توانیم از یک الگوریتم تقسیم و حل به نام جستجوی دودویی¹ استفاده کنیم.

¹ binary search

- الگوریتم تقسیم و حل آرایه را به دو قسمت تقسیم می‌کند. برای جستجوی مقدار x در آرایه A ، ابتدا مقدار x با عنصر وسط آرایه یعنی $A[n/2]$ مقایسه می‌شود. اگر x برابر با مقدار وسط آرایه بود، مقدار مورد نظر یافته شده است. اگر x کوچکتر از عنصر وسط آرایه بود، باید x را در نیمه اول آرایه یعنی $A[1:n/2-1]$ جستجو کنیم. در غیراینصورت باید x را در نیمه دوم آرایه یعنی $A[n/2+1:n]$ جستجو کنیم. این روند را برای زیر آرایه‌ها ادامه می‌دهیم تا یا x یافته شود یا مشخص شود که x در آرایه وجود ندارد.

- برای پیدا کردن عدد ۱۸ در آرایه زیر، الگوریتم به صورت زیر عمل می‌کند.



- بنابراین مراحل انجام جستجوی دودویی به صورت زیر است.

۱. تقسیم : برای پیدا کردن مقدار x در آرایه $A[low:high]$ قرار می‌دهیم $mid = \lfloor (low + high) / 2 \rfloor$. اگر $A[mid]$ برابر با x بود به نتیجه رسیده‌ایم در غیراینصورت آرایه را به دو قسمت $A[low:mid-1]$ و $A[mid+1:high]$ تقسیم می‌کنیم. این تقسیم تنها در صورتی می‌تواند انجام شود که low از $high$ بزرگ‌تر باشد.
۲. حل : در صورتی که مقدار x از $A[mid]$ کوچکتر بود، الگوریتم جستجو برای $A[low:mid-1]$ فراخوانی می‌شود، در غیراینصورت برای $A[mid+1:high]$ فراخوانی می‌شود.
۳. ترکیب : در گام ترکیب هیچ عملیاتی انجام نمی‌شود.

- الگوریتم جستجوی دودویی به صورت زیر است.

Algorithm Binary Search

```
function BINARYSEARCH(A, x, low, high)
1: if (low > high) then
2:   return -1
3: mid =  $\lfloor (low + high) / 2 \rfloor$ 
4: if (x == A[mid]) then
5:   return mid
6: if (x < A[mid]) then
7:   return BinarySearch (A, x, low, mid-1)
8: else
9:   return BinarySearch (A, x, mid+1, high)
```

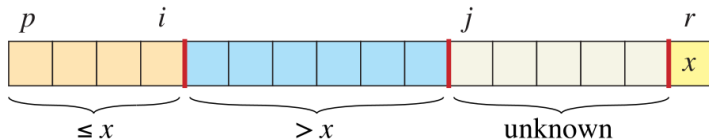
- برای جستجوی مقدار x جستجوی دودویی باید به صورت $\text{BinarySearch}(A, x, 1, n)$ فراخوانی شود.

- در تقسیم یک آرایه به دو قسمت صرفاً یک عملیات تقسیم انجام می‌شود. بنابراین $D(n) = O(1)$. تقسیم آرایه در زمان ثابت انجام می‌شود.
- بنابراین زمان اجرای الگوریتم جستجوی دودویی برای آرایه با n عنصر برابر است با زمان اجرای الگوریتم برای آرایه‌ای با $n/2$ عنصر به علاوه یک زمان ثابت.
- می‌توانیم بنویسیم $T(n) = T(\frac{n}{2}) + O(1)$ و $T(1) = O(1)$
- با حل این رابطه بازگشتی به دست می‌آوریم $T(n) = O(\lg n)$.

- یکی از الگوریتم‌های مرتب‌سازی بسیار پر استفاده الگوریتم مرتب‌سازی سریع¹ است.
- این الگوریتم یک الگوریتم تقسیم و حل است. زمان اجرای آن در بدترین حالت $\Theta(n^2)$ است، اما در حالت میانگین در زمان $\Theta(n \lg n)$ اجرا می‌شود. این الگوریتم به حافظه اضافی نیاز ندارد.
- مرتب‌سازی سریع به این صورت عمل می‌کند که یک عنصر را به عنوان عنصر محوری در نظر می‌گیرد. سپس همه عناصری که مقدار آنها از عنصر محوری کوچک‌تر است را در سمت چپ عنصر محوری و همه عناصری که مقادیرشان بیشتر از عنصر محوری است را در سمت راست آن قرار می‌دهد. سپس عناصر قسمت سمت چپ (قسمت پایین) و سمت راست (قسمت بالا) را با استفاده از مرتب‌سازی سریع مرتب می‌کند تا کل آرایه مرتب شود.

¹ quicksort algorithm

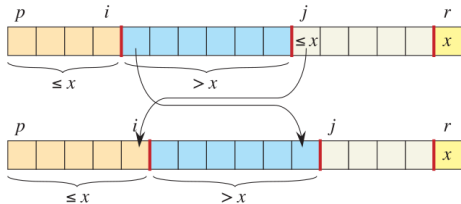
- در تقسیم آرایه به دو قسمت پایین و بالا، فرض کنید قسمت کرمی رنگ در شکل زیر عناصری باشند که مقدار آنها از عنصر محوری x کمتر و قسمت آبی رنگ عناصری باشند که مقدار آنها از عنصر محوری بیشتر است.



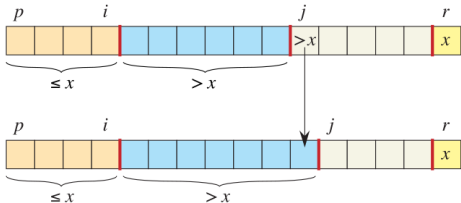
- اندیس i مرز بین قسمت پایین و قسمت بالا را نگهداری می‌کند. اندیس j مرز بین عناصری را که هنوز مکان آنها در آرایه مشخص نشده و قسمت بالا را تعیین می‌کند.

مرتب‌سازی سریع

- عناصری که هنوز مکان آنها تعیین نشده است، یک به یک بررسی می‌شوند. در صورتی که مقدار آنها از عنصر محوری x کمتر باشد به صورت زیر به قسمت پایین منتقل می‌شوند و مرز قسمت پایین و بالا تغییر می‌کند.

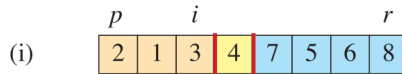
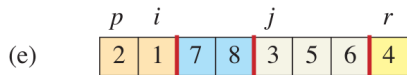
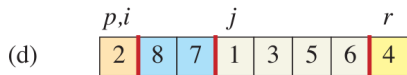
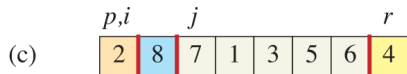
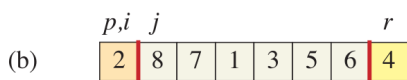
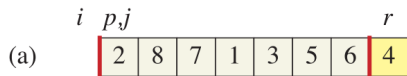


- در غیر اینصورت، آن عنصر در مکان خود نگه داشته می‌شوند.



مرتب‌سازی سریع

- در شکل زیر نحوه اجرای الگوریتم تقسیم‌بندی نشان داده شده است. عنصر محور در اینجا برابر است با $A[r]$.



مرتب‌سازی سریع

- برای مرتب‌سازی آرایه $A[p:r]$ این الگوریتم از روش تقسیم و حل به صورت زیر استفاده می‌کند.

۱. تقسیم : آرایه $A[p:r]$ به دو قسمت $A[p:q-1]$ (قسمت پایین) ¹ و $A[q+1:r]$ (قسمت بالا) ² تقسیم می‌شود به طوری که همه عناصر قسمت پایین از عنصر $A[q]$ (عنصر محور) ³ کوچکتر یا برابرند و عناصر قسمت بالا از عنصر محور بزرگ‌ترند.

۲. حل : الگوریتم مرتب‌سازی سریع برای دو زیر آرایه $A[p:q-1]$ و $A[q+1:r]$ فراخوانی می‌شود.

۳. ترکیب : در این قسمت هیچ عملیاتی انجام نمی‌شود. از آنجایی که همه عناصر $A[p:q-1]$ مرتب شده و $A[q]$ مساوی $A[q]$ هستند و همه عناصر $A[q+1:r]$ مرتب شده و بزرگتر از $A[q]$ هستند، بنابراین کل آرایه $A[p:r]$ مرتب شده است.

¹ low side

² high side

³ pivot

- الگوریتم مرتب‌سازی سریع به صورت زیر است.

Algorithm Quicksort

```
function QUICKSORT(A, p, r)
1: if p < r then ▷ Partition the subarray around the pivot, which ends up
   in A[q].
2:   q = Partition (A, p, r)
3:   Quicksort (A, p, q-1)   ▷ recursively sort the low side
4:   Quicksort (A, q+1, r)   ▷ recursively sort the high side
```

- الگوریتم تقسیم‌بندی¹ باید عناصر آرایه را به گونه‌ای جابجا کند که همهٔ عناصر قسمت پایین از عنصر محور کوچک‌تر یا مساوی و عناصر قسمت بالا از عنصر محور بزرگ‌تر باشند.

¹ partition

- الگوریتم تقسیم بندی به صورت زیر است.

Algorithm Partition

```
function PARTITION(A, p, r)
1: x = A[r]    ▷ the pivot
2: i = p - 1   ▷ highest index into the low side
3: for j = p to r-1 do ▷ process each element other than the pivot
4:     if A[j] <= x then    ▷ does this element belong on the low side?
5:         i = i+1          ▷ index of a new slot in the low side
6:         exchange A[i] with A[j]    ▷ put this element there
7: exchange A[i+1] with A[r] ▷ pivot goes just to the right of the low
   side
8: return i+1  ▷ new index of the pivot
```

- زمان اجرای الگوریتم مرتب‌سازی سریع به نحوه تقسیم‌بندی آرایه بستگی دارد. اگر تقسیم‌بندی آرایه متوازن نباشد الگوریتم در زمان $\Theta(n^2)$ اجرا می‌شود اما اگر تقسیم‌بندی متوازن باشد، الگوریتم در زمان $\Theta(n \lg n)$ اجرا می‌شود.

- اگر در هر بار تقسیم‌بندی آرایه، یک قسمت $n - 1$ عنصر و قسمت دیگر 0 عنصر داشته باشد، آنگاه تقسیم‌بندی نامتوازن است. هزینه تقسیم‌بندی آرایه برابر است با $\Theta(n)$. مرتب‌سازی یک آرایه با صفر عنصر در زمان ثابت انجام می‌شود یعنی $T(0) = \Theta(1)$ بنابراین خواهیم داشت :

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n)$$

- با حل کردن این رابطه بازگشتی به دست می‌آوریم $T(n) = \Theta(n^2)$.
- بنابراین در بدترین حالت الگوریتم مرتب‌سازی سریع مانند مرتب‌سازی درجی عمل می‌کند. بدترین حالت در مرتب‌سازی سریع وقتی رخ می‌دهد که آرایه کاملاً مرتب باشد.

- اگر الگوریتم تقسیم‌بندی، آرایه را به دو قسمت مساوی تقسیم کند، آنگاه می‌توانیم زمان اجرای الگوریتم را با استفاده از رابطه بازگشتی زیر محاسبه کنیم.

$$T(n) = 2T(n/2) + \Theta(n)$$

- با حل این رابطه به دست می‌آوریم $T(n) = \Theta(n \lg n)$.
- می‌توان اثبات کرد که الگوریتم مرتب‌سازی سریع در حالت میانگین در زمان $\Theta(n \lg n)$ اجرا می‌شود. حالت میانگین وقتی است که در الگوریتم تقسیم‌بندی، آرایه به طور میانگین با یک نسبت معین به دو قسمت تقسیم شود.

- همچنین برای اینکه بدترین حالت اتفاق نیافتد، می‌توان عنصر محوری را به صورت تصادفی انتخاب کرد و اثبات کرد که در این صورت زمان اجرای الگوریتم مرتب‌سازی سریع $\Theta(n \lg n)$ خواهد بود.
- یک روش دیگر برای اینکه بدترین حالت اتفاق نیافتد این است که بین اولین عنصر، آخرین عنصر، و عنصر وسط از آرایه، عنصری که مقدار آن میانهٔ دو مقدار دیگر است را به عنوان عنصر محوری انتخاب کنیم. این روش به استراتژی انتخاب میانهٔ سه مقدار¹ معروف است.

¹ median of three values

ضرب ماتریس‌ها

- از روش تقسیم و حل می‌توانیم برای ضرب ماتریس‌های مربعی استفاده کنیم.
- فرض کنید $A = (a_{ij})$ و $B = (b_{ij})$ دو ماتریس $n \times n$ باشند. ماتریس $C = A \cdot B$ نیز یک ماتریس $n \times n$ است که درایه‌های آن به صورت زیر محاسبه می‌شوند.
$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

- الگوریتم ضرب دو ماتریس در زیر نوشته شده‌است.

Algorithm Matrix

```
function MATRIX-MULTIPLY(A, B, C, n)
1: for i = 1 to n do ▷ compute entries in each of n rows
2:   for j = 1 to n do   ▷ compute n entries in row i
3:     for k = 1 to n do
4:       c[i,j] = c[i,j] + a[i,k] * b[k,j] ▷ compute c[i,j]
```

- از آنجایی که خط ۴ باید n^3 بار تکرار شود، بنابراین زمان مورد نیاز برای اجرای این الگوریتم برابر است با $\Theta(n^3)$.

- حال می‌خواهیم ضرب دو ماتریس را توسط روش تقسیم و حل محاسبه کنیم.
- در مرحله تقسیم، یک ماتریس $n \times n$ را به چهار ماتریس $n/2 \times n/2$ تقسیم می‌کنیم. برای سادگی فرض می‌کنیم n توانی از ۲ باشد و امکان تقسیم کردن آن به ۲ در فرایند الگوریتم تقسیم و حل وجود داشته باشد.

ضرب ماتریس‌ها

- با فرض اینکه هر یک از ماتریس‌های A ، B و C را به چهار قسمت تقسیم کنیم، محاسبات به صورت زیر انجام می‌شود.

$$C = A \cdot B \Rightarrow \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

- بنابراین خواهیم داشت :

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

ضرب ماتریس‌ها

- بنابراین ضرب یک جفت ماتریس $n \times n$ را به ضرب هشت جفت ماتریس $n/2 \times n/2$ تبدیل کردیم.
- توجه کنید که در این محاسبات نتیجه ضرب $A_{11} \cdot B_{11}$ و همچنین $A_{12} \cdot B_{21}$ باید در C_{11} ذخیره شود.

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

- حالت پایه در این الگوریتم وقتی است که می‌خواهیم دو ماتریس 1×1 را در هم ضرب کنیم که در این حالت در واقع دو عدد را در هم ضرب می‌کنیم.

- الگوریتم تقسیم و حل برای ضرب دو ماتریس را می‌توانیم به صورت زیر بنویسیم.

Algorithm Matrix

```
function MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)
```

```
    ▷ Base case.
```

```
1: if  $n==1$  then
```

```
2:    $c_{11} = c_{11} + a_{11} * b_{11}$ 
```

```
3:   return
```

```
    ▷ Divide.
```

```
4: Partition A, B, and C into  $n/2 \times n/2$  submatrices
```

```
    $A_{11}, A_{12}, A_{21}, A_{22};$ 
```

```
    $B_{11}, B_{12}, B_{21}, B_{22};$ 
```

```
    $C_{11}, C_{12}, C_{21}, C_{22};$  respectively
```

Algorithm Matrix

— ▷ Conquer.

- 5: Matrix-Multiply-Recursive ($A_{11}, B_{11}, C_{11}, n/2$)
 - 6: Matrix-Multiply-Recursive ($A_{11}, B_{12}, C_{12}, n/2$)
 - 7: Matrix-Multiply-Recursive ($A_{21}, B_{11}, C_{21}, n/2$)
 - 8: Matrix-Multiply-Recursive ($A_{21}, B_{12}, C_{22}, n/2$)
 - 9: Matrix-Multiply-Recursive ($A_{12}, B_{21}, C_{11}, n/2$)
 - 10: Matrix-Multiply-Recursive ($A_{12}, B_{22}, C_{12}, n/2$)
 - 11: Matrix-Multiply-Recursive ($A_{22}, B_{21}, C_{21}, n/2$)
 - 12: Matrix-Multiply-Recursive ($A_{22}, B_{22}, C_{22}, n/2$)
-

ضرب ماتریس‌ها

- یک ضرب ماتریسی با اندازه n را به ۸ ضرب ماتریسی با اندازه $n/2$ تبدیل کردیم.
- فرض می‌کنیم که در تقسیم ماتریس به ماتریس‌های کوچک‌تر، تنها اندیس‌ها را نامگذاری مجدد می‌کنیم و بنابراین عملیات در زمان ثابت می‌تواند انجام شود.
- بنابراین برای تحلیل این الگوریتم، می‌توانیم از رابطه بازگشتی زیر استفاده کنیم.

$$T(n) = 8T(n/2) + \Theta(1)$$

- با حل کردن این معادله به دست می‌آوریم $T(n) = \Theta(n^3)$.
- بنابراین روش تقسیم و حل زمان محاسبات را در ضرب ماتریسی کاهش نمی‌دهد.
- با استفاده از الگوریتم استراسن¹ که از یک الگوریتم تقسیم و حل بهینه‌تر استفاده می‌کند، زمان محاسبات کاهش پیدا خواهد کرد.

¹ Strassen

الگوریتم استراسن

- ضرب دو ماتریس $n \times n$ را در زمان کمتر از n^3 نیز می‌توان انجام داد. از آنجایی که برای ضرب دو ماتریس مربعی با اندازه n دقیقاً به n^3 گام محاسباتی نیاز است، بسیاری بر این باور بودند که ضرب ماتریسی نمی‌تواند در زمانی کمتر صورت بگیرد تا اینکه در سال ۱۹۶۹ ولکر استراسن^۱ ریاضیدان آلمانی، الگوریتمی با زمان اجرای $\Theta(n^{\lg 7})$ ابداع کرد. از آنجایی که $\lg 7 = 2.8073\dots$ ، بنابراین می‌توان گفت الگوریتم استراسن در زمان $O(n^{2.81})$ ضرب دو ماتریس را محاسبه می‌کند.
- الگوریتم استراسن یک الگوریتم از نوع و تقسیم و حل است.
- استراسن مجدداً در سال ۱۹۸۶ الگوریتمی از مرتبه $O(n^{2.48})$ ارائه داد. در سال ۱۹۹۰ الگوریتمی از مرتبه $O(n^{2.38})$ و در سال ۲۰۲۳ یک الگوریتم بهبود یافته ارائه شد. جستجو برای پیدا کردن سریع‌ترین الگوریتم ضرب ماتریسی همچنان ادامه دارد.

^۱ Volker Strassen

- ایده الگوریتم استراسن این است که در مراحل تقسیم و ترکیب از عملیات بیشتری استفاده می‌کند و بنابراین مراحل تقسیم و ترکیب در این الگوریتم نسبت به مراحل تقسیم و ترکیب در الگوریتم تقسیم و حل عادی زمان بیشتری صرف می‌کند ولی در ازای این افزایش زمان، در مرحله حل بازگشتی زمان کمتری مصرف می‌شود. در واقع در مرحله بازگشتی به جای فراخوانی ۸ تابع بازگشتی ۷ تابع بازگشتی فراخوانی می‌شوند.
- به عبارت دیگر عملیات مورد نیاز برای یکی از فراخوانی‌های بازگشتی توسط تعدادی عملیات جمع در مراحل تقسیم و ترکیب انجام می‌شود.

- به عنوان مثال فرض کنید می‌خواهیم به ازای دو عدد دلخواه x و y ، مقدار $x^2 - y^2$ را محاسبه کنیم. اگر بخواهیم این محاسبات را به صورت معمولی انجام دهیم، باید ابتدا x و y را به توان ۲ برسانیم و سپس دو مقدار به دست آمده را از هم کم کنیم. اما یک روش دیگر برای این محاسبات وجود دارد.
- می‌دانیم $x^2 - y^2 = (x + y)(x - y)$ ، بنابراین می‌توانیم این محاسبات را با یک عمل ضرب و دو عمل جمع و تفریق انجام دهیم. اگر x و y دو عدد باشند، زمان انجام محاسبات تفاوت چندانی نخواهد کرد، اما اگر x و y دو ماتریس بزرگ باشند، یک عمل ضرب کمتر بهبود زیادی در زمان اجرا ایجاد می‌کند.
- توجه کنید که جمع دو ماتریس مربعی با اندازه n در زمان $O(n^2)$ انجام می‌شود، و ضرب دو ماتریس در زمان $O(n^3)$.

الگوریتم استراسن

- حال که با ایده الگوریتم استراسن آشنا شدیم، الگوریتم را بررسی می‌کنیم.
- ۱. اگر $n = 1$ ، آنگاه هر ماتریس تنها یک درایه دارد. در این صورت باید یک عملیات ضرب ساده انجام داد که در زمان $\Theta(1)$ امکان پذیر است. اگر $n \neq 1$ ، آنگاه هر یک از ماتریس‌های ورودی A و B را به چهار ماتریس $n/2 \times n/2$ تقسیم می‌کنیم. این عملیات نیز در $\Theta(1)$ امکان پذیر است.
- ۲. با استفاده از زیر ماتریس‌های به دست آمده از مرحله قبل تعداد 10 ماتریس $S_1, S_2, \dots, S_{10}$ محاسبه می‌شوند. این عملیات در زمان $\Theta(n^2)$ انجام می‌شود.
- ۳. تابع ضرب ماتریسی به تعداد 7 بار بر روی ماتریس‌های S_i, A_{ij}, B_{ij} که ابعاد هر کدام $n/2 \times n/2$ است، به طور بازگشتی انجام می‌شود. نتیجه این محاسبات در 7 ماتریس $P_1, P_2, \dots, P_7$ ذخیره می‌شود. عملیات این مرحله در زمان $7T(n/2)$ انجام می‌شود.
- ۴. با استفاده از ماتریس‌های $P_1, P_2, \dots, P_7$ ، ماتریس‌های $C_{11}, C_{12}, C_{21}, C_{22}$ محاسبه می‌شود. این عملیات نیز در زمان $\Theta(n^2)$ انجام می‌شود.

الگوریتم استراسن

- بنابراین زمان کل مورد نیاز برای الگوریتم استراسن از رابطه بازگشتی زیر به دست می‌آید.

$$T(n) = 7T(n/2) + \Theta(n^2)$$

- با حل این رابطه بازگشتی به دست می‌آوریم :

$$T(n) = \Theta(n^{\lg 7}) = O(n^{2.81})$$

- حال ببینیم ماتریس‌های P_k چگونه با استفاده از ماتریس‌های A_{ij} و B_{ij} محاسبه می‌شوند.

الگوریتم استراسن

- در مرحله اول تعداد ۱۰ ماتریس S_i به صورت زیر محاسبه می‌شوند.

$$S_1 = B_{12} - B_{22}$$

$$S_2 = A_{11} + A_{12}$$

$$S_3 = A_{21} + A_{22}$$

$$S_4 = B_{21} - B_{11}$$

$$S_5 = A_{11} + A_{22}$$

$$S_6 = B_{11} + B_{22}$$

$$S_7 = A_{12} - A_{22}$$

$$S_8 = B_{21} + B_{22}$$

$$S_9 = A_{11} - A_{21}$$

$$S_{10} = B_{11} + B_{12}$$

- در محاسبات فوق 10 بار ماتریس‌های $n/2 \times n/2$ را با هم جمع کردیم که این عملیات در زمان $\Theta(n^2)$ امکان پذیر است.

- در مرحله بعد γ ماتریس P_i را با استفاده از زیر ماتریس‌های A_{ij} و B_{ij} و ماتریس‌های S_i بدست می‌آوریم.

$$P_1 = A_{11} \cdot S_1 (= A_{11} \cdot B_{12} - A_{11} \cdot B_{22})$$

$$P_2 = S_2 \cdot B_{22} (= A_{11} \cdot B_{22} + A_{12} \cdot B_{22})$$

$$P_3 = S_3 \cdot B_{11} (= A_{21} \cdot B_{11} + A_{22} \cdot B_{11})$$

$$P_4 = A_{22} \cdot S_4 (= A_{22} \cdot B_{21} - A_{22} \cdot B_{11})$$

$$P_5 = S_5 \cdot S_6 (= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22})$$

$$P_6 = S_7 \cdot S_8 (= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22})$$

$$P_7 = S_9 \cdot S_{10} (= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12})$$

الگوریتم استراسن

- بنابراین در اینجا به ۷ عملیات ضرب نیاز داریم که به صورت بازگشتی انجام می‌شوند.
- در مرحله آخر باید زیر ماتریس‌های C_{ij} را با استفاده از ماتریس‌های P_i به دست آوریم.
- این محاسبات به صورت زیر انجام می‌شوند.

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

- با بسط دادن این روابط می‌توانیم C_{ij} ها را بر اساس A_{ij} و B_{ij} ها به دست آوریم و نشان دهیم که عملیات ضرب به درستی انجام می‌شود.

$$C_{11} = P_5 + P_4 - P_2 + P_6 (= A_{11} \cdot B_{11} + A_{12} \cdot B_{21})$$

$$C_{12} = P_1 + P_2 (= A_{11} \cdot B_{12} + A_{12} \cdot B_{22})$$

$$C_{21} = P_3 + P_4 (= A_{21} \cdot B_{11} + A_{22} \cdot B_{21})$$

$$C_{22} = P_5 + P_1 - P_3 - P_7 (= A_{21} \cdot B_{12} + A_{22} \cdot B_{22})$$

- در مرحله آخر تنها از عملیات جمع استفاده می‌کنیم بنابراین محاسبه C_{ij} ها در زمان $\Theta(n^2)$ انجام می‌پذیرد.

- دقت کنید که در این روش در صورتی که تعداد سطرها یا ستونهای یک ماتریس فرد باشند، می توان یک سطر با مقادیر صفر و یا یک ستون با مقادیر صفر به ماتریس اضافه کرد و پس از انجام عملیات ضرب سطر و ستون با مقادیر صفر را حذف کرد.

- می‌خواهیم حاصلضرب دو عدد u و v را محاسبه کنیم.
- با استفاده از روش تقسیم و حل، هریک از اعداد را به دو قسمت تقسیم کرده و با استفاده از حاصل ضرب قسمت‌های کوچک‌تر، ضرب دو عدد را محاسبه می‌کنیم.
- اگر عدد u یک عدد n رقمی باشد می‌توانیم بنویسیم :

$$u = x \times 10^m + y$$

به طوری که $m = \lfloor \frac{n}{2} \rfloor$ است و x یک عدد $\lceil \frac{n}{2} \rceil$ رقمی و y یک عدد $\lfloor \frac{n}{2} \rfloor$ رقمی است.

- اگر دو عدد n رقمی u و v داشته باشیم، می‌توانیم بنویسیم :

$$u = x \times 10^m + y$$

$$v = w \times 10^m + z$$

- ضرب این دو عدد برابر است با :

$$\begin{aligned} uv &= (x \times 10^m + y)(w \times 10^m + z) \\ &= xw \times 10^{2m} + (xz + yw) \times 10^m + yz \end{aligned}$$

- حال برای ضرب دو عدد با اندازه n باید ۴ ضرب بر روی اعدادی با اندازه $\frac{n}{2}$ انجام دهیم.

- عملیات جمع در زمان خطی انجام می‌شود، بنابراین پیچیدگی زمانی ضرب دو عدد با استفاده از تقسیم و حل برابر است با :

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

- با حل این رابطه به دست می‌آوریم $T(n) = \Theta(n^2)$.

- پیچیدگی زمانی الگوریتم ضرب دو عدد n رقمی $O(n^2)$ است زیرا تعداد n^2 عملیات ضرب باید انجام شود.
- بنابراین با روش تقسیم و حل هیچ بهبودی در پیچیدگی زمانی الگوریتم ضرب حاصل نشده است.
- در روش تقسیم و حل، ضرب دو عدد n رقمی با استفاده از ۴ ضرب اعداد $\frac{n}{2}$ رقمی انجام شد. اگر بتوانیم ضرب‌ها را کاهش دهیم، می‌توانیم پیچیدگی زمانی الگوریتم را بهبود دهیم.

- توجه کنید که برای محاسبه ضرب دو عدد نیاز به محاسبه xw ، $(xz + yw)$ و yz داشتیم که برای محاسبه آن چهار عملیات ضرب نیاز بود.
- به جای انجام چهار ضرب می‌توانیم با استفاده از یک عمل ضرب مقدار r را به صورت زیر محاسبه کنیم.

$$r = (x + y)(w + z) = xw + (xz + yw) + yz$$

- بنابراین مقدار $xz + yw$ را می‌توانیم به صورت زیر محاسبه کنیم.

$$xz + yw = r - xw - yz$$

- پس برای محاسبه سه مقدار xw ، $xz + yw$ و yz نیاز داریم سه عملیات ضرب $(x + y)(w + z)$ و xw و yz را انجام دهیم.
- پیچیدگی زمانی این الگوریتم برابر است با :

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

- با محاسبه این رابطه بازگشتی به دست می آوریم :

$$T(n) = O(n^{\lg 3}) = O(n^{1.58})$$

- این الگوریتم در سال ۱۹۶۰ توسط آناتولی کاراستوبا^۱ ریاضی‌دان روسی ابداع شد و الگوریتم کاراتسوبا نامیده می‌شود.
- در سال ۲۰۱۹ یک الگوریتم از مرتبه $O(n \lg n)$ برای ضرب اعداد معرفی شده است که بر پایه الگوریتم شونهاج-استراسن^۲ و تبدیل‌های فوریه سریع بنا نهاده شده است.

^۱ Anatoly Karatsuba

^۲ Schonhage-Strassen algorithm

حل روابط بازگشتی

- در مسائل تقسیم و حل دیدیم چگونه می‌توان از روابط بازگشتی برای محاسبهٔ زمان اجرای الگوریتم‌ها بهره گرفت. در اینجا چند روش برای حل روابط بازگشتی مطرح می‌کنیم که عبارتند از روش جایگذاری¹، روش درخت بازگشت² و روش قضیه اصلی³.

¹ substitution method

² recursion-tree method

³ master theorem method

- روش جایگذاری برای حل روابط بازگشتی از دو گام تشکیل شده است. در گام اول جواب رابطه بازگشتی یا عبارت فرم بسته¹ که در رابطه بازگشتی صدق می‌کند حدس زده می‌شود. در گام دوم توسط استقرای ریاضی² اثبات می‌شود که جوابی که حدس زده شده است درست است و در رابطه بازگشتی صدق می‌کند.
- برای اثبات توسط استقرای ریاضی، ابتدا باید ثابت کرد که جواب حدس زده شده برای مقادیر کوچک n درست است. سپس باید اثبات کرد که اگر جواب حدس زده شده برای n درست باشد، برای $n+1$ نیز درست است. در این روش از جایگذاری جواب حدس زده شده در رابطه اصلی برای اثبات استفاده می‌شود و به همین دلیل روش جایگذاری نامیده می‌شود.
- متأسفانه هیچ قاعده کلی برای حدس زدن جواب رابطه بازگشتی وجود ندارد و یک حدس خوب به کمی تجربه و خلاقیت نیاز دارد.

¹ closed-form expression

² mathematical induction

- برای مثال فرض کنید می‌خواهیم رابطه $T(n) = 2T(n - 1)$ و $T(0) = 1$ را حل کنیم.
- این رابطه را برای n های کوچک می‌نویسیم و حدس می‌زنیم $T(n) = 2^n$ باشد.
- سپس رابطه را با استفاده از استقرا اثبات می‌کنیم.

- در برخی مواقع یک رابطه بازگشتی شبیه رابطه‌هایی است که جواب آنها را می‌دانیم و در چنین مواقعی می‌توانیم از حدس استفاده کنیم.
- برای مثال رابطه $T(n) = 2T(n/2 + 17) + \Theta(n)$ را در نظر بگیرید. شبیه این رابطه را بدون عدد ۱۷ قبلاً دیده‌ایم اما می‌توانیم حدس بزنیم که این عدد برای n های بزرگ تأثیر زیادی ندارد. پس حدس می‌زنیم که جواب این رابطه $T(n) = O(n \lg n)$ باشد.

روش درخت بازگشت

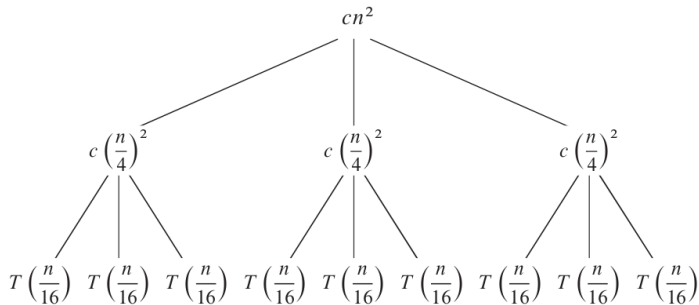
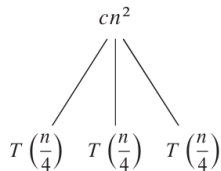
- روش دیگر برای حل مسائل بازگشتی، استفاده از درخت بازگشت¹ است.
- در این روش هر رأس از درخت، هزینه محاسبات یکی از زیر مسئله‌ها را نشان می‌دهد.
- هزینه کل اجرای یک برنامه عبارت است از هزینه‌ای که در سطح صفر درخت برای تقسیم و ترکیب نیاز است به علاوه هزینه محاسبه زیر مسئله‌ها. به همین ترتیب هزینه محاسبه هر یک از زیر مسئله‌های سطح اول تشکیل می‌شود از هزینه تقسیم و ترکیب به علاوه هزینه زیر مسئله‌های سطح دوم و به همین ترتیب الی آخر.
- بنابراین اگر هزینه محاسبه همه رئوس درخت بازگشت را جمع کنیم، هزینه کل اجرای برنامه به دست می‌آید.

¹ recursion tree

– یک مثال درخت بازگشت را در اینجا بررسی می‌کنیم. رابطه بازگشتی $T(n) = 3T(n/4) + \Theta(n^2)$ را در نظر بگیرید.

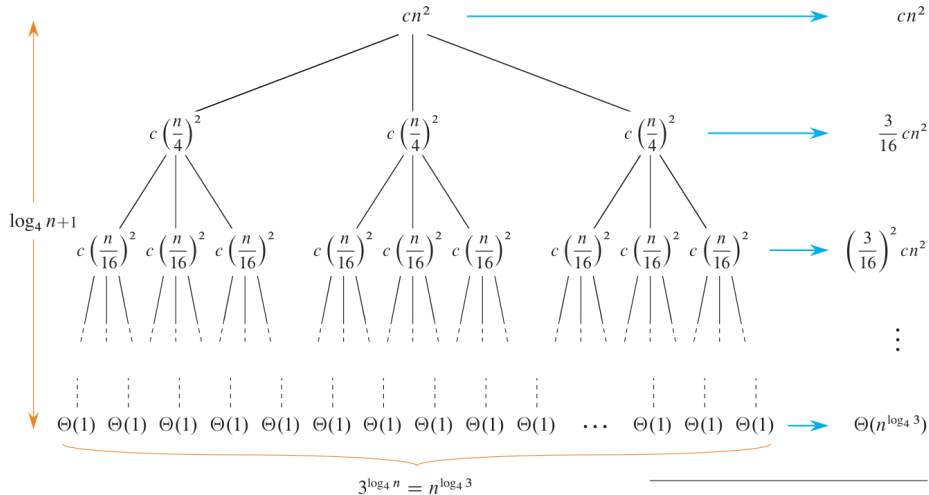
- شکل زیر تشکیل درخت بازگشت را برای این رابطه بازگشتی در دو مرحله اول نشان می‌دهد.

$T(n)$



روش درخت بازگشت

- اگر مجموع هزینه‌ها را در هر سطح محاسبه کنیم، درختی با هزینه‌های قید شده در زیر خواهیم داشت.



– سپس هزینه‌های سطوح این درخت بازگشت را با هم جمع می‌کنیم و جواب رابطه بازگشتی را به دست می‌آوریم.

$$\begin{aligned}
 T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{(\log_4^n)-1} cn^2 + \Theta(n^{\log_4^3}) \\
 &= \sum_{i=0}^{(\log_4^n)-1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4^3}) \\
 &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4^3}) \\
 &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4^3}) \\
 &= \frac{16}{13} cn^2 + \Theta(n^{\log_4^3}) \\
 &= O(n^2) \qquad (\Theta(n^{\log_4^3}) = O(n^{0.8}) = O(n^2)).
 \end{aligned}$$

روش قضیه اصلی

- روش قضیه اصلی¹ برای حل مسائل بازگشتی استفاده می‌شود که به صورت $T(n) = aT(n/b) + f(n)$ هستند به طوری که $a > 0$ و $b > 1$ دو ثابت هستند.
- تابع $f(n)$ در اینجا تابع محرک² نامیده می‌شود و یک رابطه بازگشتی که به شکل مذکور است، رابطه بازگشتی اصلی³ نامیده می‌شود.
- در واقع رابطه بازگشتی اصلی زمان اجرای الگوریتم‌های تقسیم و حل را توصیف می‌کند که مسئله‌ای به اندازه n را به a زیر مسئله هر کدام با اندازه n/b تقسیم می‌کنند. تابع $f(n)$ هزینه تقسیم مسئله به زیر مسئله‌ها به علاوه هزینه ترکیب زیر مسئله‌ها را نشان می‌دهد.
- اگر یک رابطه بازگشتی شبیه رابطه قضیه اصلی باشد و علاوه بر آن چند عملگر کف و سقف در آن وجود داشته باشد، همچنان می‌توان از رابطه قضیه اصلی استفاده کرد.

¹ master theorem method

² driving function

³ master recurrence

- قضیه اصلی : فرض کنید $a > 0$ و $b > 1$ دو ثابت باشند و $f(n)$ یک تابع باشد که برای اعداد بسیار بزرگ تعریف شده باشد.
- رابطه بازگشتی $T(n)$ که بر روی اعداد طبیعی $n \in \mathbb{N}$ تعریف شده است را به صورت زیر در نظر بگیرید.
$$T(n) = aT(n/b) + f(n)$$

- رفتار مجانبی $T(n) = aT(n/b) + f(n)$ به صورت زیر است :

۱- اگر ثابت $\epsilon > 0$ وجود داشته باشد به طوری که $f(n) = O(n^{\log_b^a - \epsilon})$ آنگاه $T(n) = \Theta(n^{\log_b^a})$.

۲- اگر ثابت $k \geq 0$ وجود داشته باشد به طوری که $f(n) = \Theta(n^{\log_b^a} \lg^k n)$ آنگاه $T(n) = \Theta(n^{\log_b^a} \lg^{k+1} n)$.

۳- اگر ثابت $\epsilon > 0$ وجود داشته باشد به طوری که $f(n) = \Omega(n^{\log_b^a + \epsilon})$ آنگاه $T(n) = \Theta(f(n))$.
 برای برخی از توابع $f(n)$ نیاز داریم بررسی کنیم $f(n)$ در رابطه $af(n/b) \leq cf(n)$ به ازای $c < 1$ و n های به اندازه کافی بزرگ صدق کند، اما برای توابعی که در تحلیل الگوریتم ها به آنها برمیخوریم این شرط معمولا برقرار است.

- ۱- در حالت اول رشد جزء بازگشتی از رشد تابع محرک بیشتر است. به عنوان مثال در $T(n) = 2T(n/2) + \lg n$ رشد جزء بازگشتی $\Theta(n)$ و رشد تابع محرک $\Theta(\lg n)$ است. بنابراین $T(n) = \Theta(n)$.
- ۲- در حالت دوم رشد جزء بازگشتی و تابع محرک برابر است و یا رشد تابع محرک با یک ضریب $\Theta(\lg^k n)$ از جزء بازگشتی سریع تر است. به عنوان مثال در $T(n) = 2T(n/2) + n \lg n$ رشد جزء بازگشتی $\Theta(n)$ و رشد تابع محرک $\Theta(n \lg n)$ است. در این حالت $T(n) = \Theta(n \lg^2 n)$.
- ۳- در حالت سوم رشد جزء بازگشتی از رشد تابع محرک کمتر است. به عنوان مثال در $T(n) = 2T(n/2) + n^2$ رشد جزء بازگشتی $\Theta(n)$ و رشد تابع محرک $\Theta(n^2)$ است. بنابراین $T(n) = \Theta(n^2)$.

- در یک حالت خاص اگر داشته باشیم، $T(n) = aT(n/b) + cn^k$ آنگاه می‌توانیم اثبات کنیم:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{اگر } a > b^k \\ \Theta(n^k \lg n) & \text{اگر } a = b^k \\ \Theta(n^k) & \text{اگر } a < b^k \end{cases}$$

- رابطه بازگشتی $T(n) = 9T(n/3) + n$ را در نظر بگیرید. در این رابطه داریم $a = 9$ و $b = 3$ بنابراین به دست می‌آوریم $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. از آنجایی که $f(n) = n = O(n^{2-\epsilon})$ به ازای هر ثابت $\epsilon < 1$ بنابراین می‌توانیم حالت اول در قضیه اصلی را در نظر بگیریم و نتیجه بگیریم $T(n) = \Theta(n^2)$.

- رابطه بازگشتی $T(n) = T(2n/3) + 1$ را در نظر بگیرید. در این رابطه داریم $a = 1$ و $b = 3/2$ بنابراین $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$ در اینجا حالت دوم در قضیه اصلی را داریم یعنی $f(n) = 1 = \Theta(n^{\log_b a} \lg^0 n) = \Theta(1)$ بنابراین جواب رابطه بازگشتی برابر است با $T(n) = \Theta(\lg n)$.

- در رابطه بازگشتی $T(n) = 3T(n/4) + n \lg n$ داریم $a = 3$ و $b = 4$ که بدین معنی است که $n^{\log_b a} = n^{\log_4 3} = \Theta(n^{0.793})$. از آنجایی که $f(n) = n \lg n = \Omega(n^{\log_4 3 + \epsilon})$ جایی که ϵ حدود 0.2 است، بنابراین حالت سوم در قضیه اصلی را می‌توانیم در نظر بگیریم اگر شرط $af(n/b) \leq cf(n)$ برقرار باشد.

$$af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = 3/4f(n)$$

بنابراین با استفاده از حالت سوم جواب رابطه بازگشتی برابر است با $T(n) = \Theta(n \lg n)$.

- رابطه بازگشتی $T(n) = 2T(n/2) + \Theta(n)$ رابطه‌ای بود که برای مرتب‌سازی ادغامی به دست آوردیم. از آنجایی که $a = 2$ و $b = 2$ داریم $n^{\log_2 2} = n$. حالت دوم در اینجا برقرار است زیرا به ازای $k = 0$ داریم $f(n) = \Theta(n)$ و بنابراین جواب رابطه بازگشتی برابر است با $T(n) = \Theta(n \lg n)$.

- رابطه $T(n) = 8T(n/2) + \Theta(1)$ زمان اجرای الگوریتم ضرب ماتریسی را توصیف می‌کند. در اینجا داریم $a = 8$ و $b = 2$ بنابراین $n^{\log_2 8} = n^3$. تابع محرک $f(n) = \Theta(1)$ است و بنابراین به ازای هر $\epsilon < 3$ داریم $f(n) = O(n^{3-\epsilon})$. بنابراین حالت اول قضیه اصلی برقرار است. نتیجه می‌گیریم $T(n) = \Theta(n^3)$.

- در تحلیل زمان اجرای الگوریتم استراسن رابطه $T(n) = 7T(n/2) + \Theta(n^2)$ را به دست آوردیم. در این رابطه بازگشتی $a = 7$ و $b = 2$ بنابراین $n^{\log_2 7} = n^{\lg 7}$. از آنجایی که $\lg 7 = 2.8073\dots$ ، می‌توانیم قرار دهیم $\epsilon = 0.8$ و برای تابع محرک خواهیم داشت $f(n) = \Theta(n^2) = O(n^{\lg 7 - \epsilon})$ ، پس حالت اول در قضیه اصلی برقرار است و بنابراین جواب رابطه بازگشتی برابر است با $T(n) = \Theta(n^{\lg 7})$.